第四章　TIDE-Swarm 详细设计与实现

在第三章总体架构的基础上，本章介绍 TIDE-Anchor 上下文连续性与记忆保持、TIDE-Mesh 动态异构协作与低熵通信、TIDE-Placement 端边云推理调度三项核心机制，说明其状态模型、运行过程与核心算法，并分析计算与通信开销。三项机制通过任务状态和执行反馈衔接，共同支撑长程任务的持续推进与异常恢复。

面向赛题提出的上下文连续性、动态异构拓扑、低熵通信、神经符号协同推理与端边云调度要求，本文以**语义生成、显式校验、受控执行和证据反馈**组织长程协作。神经侧通过大模型理解自然语言意图、生成任务计划与阶段结果，并在配置质量评审模型时提供语义判断；符号侧将任务目标、依赖、资源需求和验收条件组织为结构化状态，以计划校验、资源硬过滤、消息字段保护和质量规则约束执行。两侧通过计划中间表示、反例、任务账本与结构化消息交换信息，使推理建议转化为可检查、可修订的执行依据。

三项机制在这一闭环中分别回答“依据什么继续推理”“由哪些成员协作”“在哪类资源上执行”。TIDE-Anchor 提供稳定目标与可追溯证据，TIDE-Mesh 依据当前任务、能力需求和失败反馈生成稀疏的活跃协作关系，TIDE-Placement 依据实时性、敏感度和资源状态安排任务阶段。**神经符号协同贯穿三项机制的决策接口**：语义判断提出候选，显式规则限定可行范围，运行结果再更新计划、成员选择和资源分配。第4.4节进一步说明计划校验、动态拓扑与分级恢复如何衔接，并给出可观测的评价口径。

4.1　TIDE-Anchor 超长程上下文连续性与记忆保持

4.1.1　问题定义与总体设计

长程任务中的上下文连续性，是在任务跨越多轮工具调用、角色交接和执行中断时，持续提供正确目标、当前状态和可核验证据的能力。数小时乃至数天是本机制面向的设计场景，实际持续运行能力仍需按实验时长验证。上下文窗口扩展只能增加信息容量，不能自动解决信息选择、错误隔离和恢复一致性问题。

本文围绕四类相互关联的失效组织 TIDE-Anchor。第一，上下文溢出与退化：工具结果持续累积，历史截断可能丢失关键参数和执行结果。第二，指令稀释与注意力漂移：约束虽仍在输入中，却未被当前决策有效利用；长上下文中的位置敏感性已有相关研究讨论 [1]。第三，误差累积与级联传播：未核验的阶段输出被下游当作事实，局部

故障转化为后续决策的错误前提。第四，静默置信度失效与状态丢失：服务调用成功却没有任务进展，或中断后无法一致恢复信息与执行位置。

表 4.1: 四类长程失效与 TIDE-Anchor 的对应机制

失效模式	设计机制	直接检查对象
上下文溢出与退化	状态投影、原文归档、四级记忆检索与装配后预算检查	输入用量、证据覆盖与归档可访问性
指令稀释与漂移	共享状态中的目标和约束每步重锚定，动作前后进行契约检查	约束满足、目标关联与待办进展
误差累积与级联	候选结果隔离、证据验证、依赖版本追踪与局部重执行	已验证状态、来源与下游依赖
静默失效与状态丢失	进展检测、统一状态检查点与恢复前一致性校验	实际产物变化、执行前沿及副作用记录

TIDE-Anchor 以**版本化共享状态和四级记忆**构成节点信息供给的两个来源。共享状态维护当前任务的权威视图；记忆保存已完成工作提炼出的、具有适用条件的可复用信息。二者经上下文装配器进入节点，原始工具输出和文件保留在归档中。账本与检查点统一归入共享状态管理组件：账本是当前状态的可读视图，检查点是这一状态连同执行位置的可恢复版本。

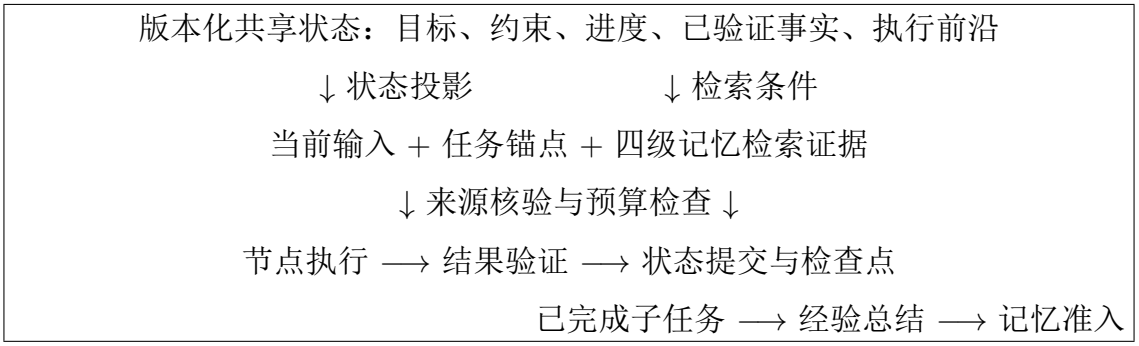


图 4.1: 共享状态与四级记忆协同的上下文供给架构

4.1.2　共享状态与四级记忆的职责边界

版本化共享状态　设第 t 个已提交版本为

$$S_t = \langle g, \mathcal{C}_t, \mathcal{P}_t, \mathcal{F}_t, \mathcal{A}_t, \mathcal{D}_t, \mathcal{X}_t, v_t \rangle, \tag{4.1}$$

其中，g 为原始目标，$\mathcal{C}_t$ 为当前有效约束，$\mathcal{P}_t$ 为待办及完成状态，$\mathcal{F}_t$ 为带验证标记的事实，$\mathcal{A}_t$ 为产物与归档引用，$\mathcal{D}_t$ 为证据和产物依赖，$\mathcal{X}_t$ 为执行前沿、预算及尝试记录，v_t 为版本。节点产物先进入候选区，通过相应验收后再更新权威字段。未通过验证的推测可以保留，但必须携带未验证或争议状态。

共享不意味着把整个状态复制给每个模型。节点只获得其权限和职责允许读取的投影 $P_n(S_t)$；原始目标、有效硬约束及本步验收条件构成任务锚点。并行节点基于明确版本读取状态，写回时检查基础版本和字段归并规则，避免后提交的旧结果覆盖更新后的约束。无法自动消解的写入冲突进入协调流程。

四级记忆的产生与使用　记忆来源于**已结束的节点操作或已完成子任务的总结**，其作用是复用经验和证据，而不是复制当前任务状态。整个任务尚未结束时，已验收子任务也可以形成 TASK 记忆；跨任务的 PROJECT 或 GLOBAL 记忆则需要更严格的归纳和准入。运行中的草稿、即时工具返回和未经检查的模型输出优先留在工作缓冲或候选结果区，不能直接作为稳定知识传播。

表 4.2: 四级记忆的产生条件与复用范围

作用域	形成条件与典型内容	读取边界
WORKING	已结束局部操作的短摘要；尚待子任务验收时标为候选	当前节点或局部执行链，短生命周期
TASK	已验收子任务的结论、参数解释和失败经验	当前运行，绑定任务版本
PROJECT	完成任务后经核验的项目约定、处理方法和历史结论	同项目跨运行，受权限和有效期约束
GLOBAL	从完成任务中抽象、去除私有信息并通过准入的通用经验	经授权跨项目使用，不携带项目秘密

四级表示使用范围，不是同一条消息逐级自动升级的流水线。记忆应记录适用条件、结论、证据引用、有效时间、来源任务和验证状态；失败经验也可保留，但应表达“何种条件下失败”，不能把失败结果写成正确事实。HOT、WARM、STRUCTURED、SEMANTIC 和 COLD 属于存储介质，与上述作用域独立配置。

4.1.3　面向上下文溢出与退化的按需装配

每个节点的上下文由当前输入、共享状态投影和记忆检索结果共同构造：

$$C_{n,t} = O_n \oplus A_n(S_t) \oplus I_{n,t} \oplus P_n^{\text{detail}}(S_t) \oplus E_{n,t}, \tag{4.2}$$

其中，O_n 为角色和输出契约，$A_n(S_t)$ 为任务锚点，$I_{n,t}$ 包含本步输入及第4.2节通信模块交付的定向消息，P_n^{detail} 为其余必要状态，$E_{n,t}$ 为从四级记忆唤醒的证据。状态投影和检索结果按来源标识去重；通信信箱不再形成独立的全量历史副本。

检索先按任务、项目、权限、时间和验证状态限定候选，再结合语义与词项匹配排序。WORKING 和 TASK 优先补充本次运行的局部依据，PROJECT 和 GLOBAL 提供条件适配的历史经验；跨层候选可统一重排。命中摘要后，仅在核对参数、变更或来

表 4.3: 共享状态、检查点、记忆与归档的关系

对象	主要问题	更新与恢复语义
共享状态当前视图	现在要完成什么，哪些结果可信	验收后按版本提交，账本呈现该视图
共享状态检查点	中断后从哪个一致版本继续	保存状态、执行前沿和必要恢复引用
四级记忆	哪些已完成工作的信息可在此复用	总结后准入，按范围检索，可失效或撤销
原文归档	如何核验摘要和阶段结论	保存工具结果及产物原文，以引用和摘要校验关联

源时读取原文。Silent、Standard、Deep 和 Recovery 是检索深度配置，分别面向局部操作、常规执行、复核与恢复，不改变权限边界。

$$B_{\text{in}} = \max(0, B_{\max} - B_{\text{out}} - B_{\text{margin}}), \qquad \text{Tok}(C_{n,t}) \le B_{\text{in}}, \tag{4.3}$$

其中 B_{margin} 为工具格式和计数误差预留空间。预算检查针对最终装配结果，而不只检查原始共享状态。超限时依次减少低相关记忆、缩短可回溯证据正文、移出冗余状态描述；目标、硬约束、必要证据引用和输出契约优先保留。若必要信息本身已超限，则暂停或拆分任务，不以静默截断破坏执行条件。

长工具结果在写回时形成摘要与原文引用，完整载荷留在归档。该设计把模型可见输入的增长限制在当前决策所需范围内，但摘要可能遗漏细节，检索也可能漏召回，因此仍需通过证据充分性及关键参数保留率检查实际效果。

4.1.4 面向指令稀释与注意力漂移的任务重锚定

任务重锚定在每步重新构造目标与约束，使它们不依赖历史检索是否命中。锚点包含原始目标、当前有效约束、本步目标、已确认事实、不可假定事项和验收条件。上下文模板把角色与约束置于明确位置，并在临近当前动作处给出简短验收清单；这种输入组织用于减少约束被过程信息淹没的机会，不构成对模型注意力的理论保证。

状态约束与历史经验冲突时，先检查二者的有效版本与来源。当前用户授权和有效任务约束决定执行边界，记忆只能提出参考或冲突线索。系统不能因历史任务曾允许某种行为，就改变本次任务的权限、资源范围或验收条件。

模型执行前，检查候选动作的工具、参数和资源是否满足显式约束；执行后，再将结果与当前验收条件核对。语义目标关联可由评审模型辅助判断，可编码的权限、数值和输出结构采用规则检查。发现偏离时生成定位明确的修订指令；连续偏离或修订预算耗尽时暂停并记录原因。该路径将重锚定与可观测检查连接起来，避免仅依赖反复复述目标。

4.1.5　面向误差累积与级联传播的证据隔离与局部修复

结果隔离以“已返回”和“已确认”分开处理。HTTP 成功、工具正常退出或模型自报高置信度只能说明一次调用完成，不能自动证明业务结果正确。节点输出先标为候选，按任务性质检查非空产物、来源、参数一致性、测试或其他验收证据；通过后提交状态，再提炼相应记忆。工具异常与低质量结果保留为失败记录，不填入成功事实字段。

依赖追踪记录节点实际使用的事实、记忆和产物版本。设结论 y 的输入依赖为

$$\mathrm{Dep}(y) = \{(x_i, v_i)\}_{i=1}^{m}. \tag{4.4}$$

当某版本被证实错误或过期时，先隔离该记录并展开原文复核，再将直接及间接依赖它的结果标为待重验。对已失效依赖的后继执行设置阻断，避免错误继续进入第4.2节的消息传输与团队交接。

A 到 B 再到 C 的修复示例　设 A 提取数据单位，B 据此计算指标，C 生成报告。B 引用历史记忆 $m^{(2)}$，将“元”误读为“万元”。若 C 在生成报告前检查到指标与原始表格不一致，先暂停依赖该指标的报告步骤，读取证据确认是记忆错误、A 的提取错误，还是 B 的计算错误。若确认只有 $m^{(2)}$ 错误，则撤销该版本，保留 A 中独立且正确的提取结果，使用核验后的单位重新执行 B，再执行 C；若错误来自 A，则 A 及受影响的 B、C 均需重做。单纯检索得分低或模型不确定时，应先补证，不能直接认定历史记忆错误。

若修复期间用户增加了新约束，旧检查点只作为重建依据，最新有效约束必须继续生效。重执行生成新的尝试标识和产物版本，旧结果保留失效标记供审计。对已发送通知、提交订单或写入外部系统的操作，先核对幂等键和外部结果；无法撤销的副作用进入补偿或人工处理路径，不把状态回滚等同于现实世界回滚。

4.1.6　面向静默失效与状态丢失的进展检测与统一恢复

以任务进展补充服务监控　静默失效检测检查连续窗口内是否出现新的有效产物、已验证事实或待办推进，并结合动作重复、相似输出、资源重复访问和质量返工记录定位停滞。无进展只是触发核验的信号：数据下载、长计算等步骤应使用心跳、进度计数或任务专用检查器，避免把正常等待误判为死循环。模型自报置信度仅作为辅助信号。

检测结果分为继续观察、补充证据、修订本步、局部重规划和暂停。状态中记录触发证据、处理动作及尝试次数；每类恢复设置有限预算。持续返回状态码 200 而重复生成相同无效产物，仍会因缺少可核验进展进入恢复路径。

统一的状态检查点　账本、图状态和执行位置在逻辑上组成一个可恢复状态单元：

$$K_t = \langle S_t, \mathcal{E}_t, \mathcal{Q}_t, \mathcal{R}_t^{\mathrm{mem}}, \mathcal{U}_t, \nu_t \rangle, \tag{4.5}$$

其中，$\mathcal{E}_t$ 为执行前沿，$\mathcal{Q}_t$ 为消息及接收版本恢复引用，$\mathcal{R}_t^{\mathrm{mem}}$ 为本次运行使用的记忆版本清单，$\mathcal{U}_t$ 为外部副作用和幂等记录，ν_t 为提交标识。无需复制整个跨项目记忆库，但必须能定位当时使用的版本及其后续撤销信息。

持久化时先写入各对象与引用，再写完成清单；只有引用完整且版本一致的清单才被选为恢复点。该协议把现有上下文与图检查点组织为统一恢复入口，底层仍可使用不同存储。并行写入和跨存储故障需要提交协调，不能仅靠相同文件名视为原子提交。

选择完整检查点 → 核对状态版本与执行前沿
→ 合并最新有效约束、失效记录与外部结果
→ 重建消息接收版本、按需唤醒记忆
→ 检查依赖、预算与资源 → 继续或局部重执行

图 4.2: 版本化共享状态的恢复与继续执行流程

4.1.7　核心算法、实现边界与验证口径

Algorithm 1: 共享状态与四级记忆驱动的节点执行

Input: 共享状态 S_t，节点 n，四级记忆 M，恢复与预算策略 b

Output: 新状态版本、恢复请求或暂停原因

读取 n 所需的状态版本、任务目标及有效约束;
根据任务、角色和权限检索记忆，核验来源、有效期与撤销状态;
构造任务锚点、状态投影、当前输入和相关记忆;
对最终上下文检查预算；必要信息超限时拆分或暂停;
执行 n，保留候选产物、输入依赖和工具记录;
if 验收通过且依赖版本仍有效 **then**
    提交状态增量；对已完成工作生成候选记忆并执行准入;
    保存一致的状态、执行位置与必要恢复引用;
else
    隔离候选产物，定位错误依赖并计算受影响范围;
    在恢复预算内补证、局部重执行或暂停;
检查实际进展并记录停滞与处理动作;

设候选记忆数为 N，展开条数为 K、单条组织长度为 R，检索成本为 $T_{\mathrm{ret}}(N)$，则完整排序路径为

$$T_{\mathrm{wake}} = T_{\mathrm{ret}}(N) + O(N \log N + KR). \tag{4.6}$$

预算计数随装配文本长度增长；失效传播在受影响依赖子图上遍历，成本为 $O(|V_D| +$

Algorithm 2: 带有效性检查的分级证据唤醒

Input: 查询 q，状态 S_t，四级记忆 M，配置 p

Output: 证据集合 E 及依赖版本清单

按权限、作用域、时间和验证状态过滤候选记忆;

收集词项与语义候选，统一重排、去重，选择至多 K 条;

对可能冲突或需要核对参数的记录展开原文;

隔离已撤销、来源失效或适用条件不满足的记录;

按剩余输入预算组织证据，保留来源与版本;

return E 及依赖版本清单;

$|E_D|$)。检查点写入开销取决于状态和增量载荷大小。以上不包含模型、原文读取及外部工具的实际耗时。

现有工程已提供账本、预算控制、分层记忆、归档、漂移检测及上下文和图检查点等基础组件。本节进一步给出统一版本清单、验收后记忆准入、细粒度依赖撤销及局部重执行的目标设计；这些扩展需要在运行时补齐后单独验证，现有组件测试不能证明整个目标架构已实现。

四类问题应分别通过上下文用量与关键证据保留、约束遵守与目标保持、错误下游采纳率与修复范围、静默停滞检出与一致恢复结果评价。仅减少输入长度不足以证明记忆有效，仅检测到异常也不足以证明已经恢复。验证应包含 A 到 B 到 C 的错误记忆注入、正常长计算、并行写入冲突、中断恢复及外部副作用核对。

4.2　TIDE-Mesh：动态异构拓扑、低熵通信与时态证据记忆

4.2.1　应用场景与设计目标

动态拓扑用于**任务所需能力、证据来源或处理路径在执行中才逐步明确**的场景。若任务步骤、参与者和验收关系均已确定，固定流水线即可满足需求。对于需要边执行边发现问题的任务，则需根据新证据决定何时增加角色、分出并行调查、加入复核或替换失效成员。

已有系统为这一需求提供了场景参照。Magentic-One 通过文件、浏览、编码和执行角色协作处理跨工具任务 [2]；产品架构文档也区分了顺序、并行和动态编排的适用条件 [3]。本文据此选择三类任务作为设计场景，表 4.4给出各自的拓扑变化原因。这些场景用于说明机制，不预设本文已经达到相关系统的实测能力。

上述任务共同要求区分两个决策：**由谁协作、形成哪些依赖关系；沿这些关系何时通信、传递什么信息**。TIDE-Mesh 因此由任务驱动的拓扑控制器和边上的语义通信架构组成。前者输出有任务版本和生命周期的协作图，后者对每条有效边提供一致的消息组织、筛选、压缩及接收校验接口。

表 4.4: 动态协作拓扑的任务场景与触发条件

场景	初始工作	运行期变化及所需关系
软件故障修复	定位问题、读取代码和日志	根因不明确时并行调查；定位后汇聚给修复角色；变更后调用测试和独立复核；工具失效时切换可行成员
研究报告生成	阅读资料、形成证据和论点	不同来源并行检索；出现矛盾时增加核验；证据充分后汇聚写作；新增问题触发补充调查
业务资料处理	从文档提取需求与业务字段	根据资料类型增加专业角色；涉及权限或条款时增加审查；人工变更要求后重建尚未完成的处理分支

4.2.2　任务图、活跃协作图与动态流转

两类图的职责　任务图 $D_t = (\mathcal{T}_t, \mathcal{L}_t)$ 表达子任务及产物依赖，活跃协作图 $G_t = (V_t, E_t)$ 表达当前由哪些 Agent 或团队承担子任务，以及哪些角色之间需要交接。候选成员池保持相对稳定，活跃图随当前任务和事件变化。外层规划、质量门和结果汇聚提供控制边界，内层图只在边界内编译和调整。

每个子任务 τ 记录输入产物、所需能力、硬性资源条件、输出契约及验收条件。控制器先确定任务关系，再分配成员，最后依据生产者与消费者关系生成通信边。每条边可写为

$$e = \langle u, v, \tau, \kappa, v_{\text{task}}, b_e, \ell_e \rangle, \tag{4.7}$$

其中 u, v 为发送和接收端，κ 为边类型，v_{task} 为任务版本，b_e 为预算，ℓ_e 为有效期。边类型包括委派、证据供给、产物交接、复核、替补及撤销通知。撤销与重构导致边失效时，应同时处理尚未消费的旧版本消息。

三种基本流线　初始信息充分且依赖明确时采用顺序交接。存在独立未知因素时，控制器生成并行分支，并在约定的汇聚节点等待必要证据。产物具有较高风险或存在争议时，再引入复核分支。并行范围和复核条件由当前任务决定，不要求每个任务都调用全部成员。

软件修复的完整示例　用户只提供“升级后接口超时”时，初始接入角色先确认环境与复现条件。若无法区分代码变化和部署配置问题，拆出代码调查与环境调查两个分支；二者仅向根因汇聚角色发送相关差异和证据。根因确定后，关闭不再需要的调查边，激活修复到测试的产物交接边。若测试发现涉及数据迁移，再增加迁移检查角色；修复产物满足条件后，由复核角色验收并交付。

任务状态与新事件 → 剩余任务图 → 能力互补选择
→ 依赖与权限检查 → 活跃协作图
顺序：定位 → 修复 → 测试
分支：代码调查 ‖ 环境调查 → 证据汇聚
复核：阶段产物 → 独立检查 → 通过或修订
每个箭头均绑定通信契约、版本、预算和生命周期

图 4.3: 从任务事件到活跃协作图的编译及典型流线

任务变更、成员失效和证据冲突分别触发剩余任务更新、失败替补和新增核验边。测试失败时，控制器把反例发回新的修订轮次，而非在同一任务版本内创建无限回环。每个版本内的产物依赖采用可执行的有向无环结构，跨轮反馈通过版本和尝试记录表达；反馈和通知通道可另行存在，但不能绕过调度依赖触发执行。

逻辑协作边与物理通道　图中的 A 到 B 表示 B 消费 A 的特定产物，不要求二者建立独立网络连接。单 Agent 和团队都通过统一通信端口暴露任务输入、消息发送和接收接口；团队内部的交接也可经 Supervisor 中介。若存在 A 到 B 的逻辑边，中介应保留二者标识及边契约，不因中转将其扩大为全员广播。

现有团队内部调用主要经 Supervisor 组织，Capsule 标准链路主要覆盖图节点及其真实后继。本节将相同端口进一步扩展到团队和内部交接边，是目标架构的一部分；需要补齐成员级端口和事件追踪后，才能声称各类协作边均已统一接入。

4.2.3　能力互补选择与事件驱动的拓扑优化

从独立排序转向团队边际贡献　按单个成员得分取 Top-K 可以快速选择候选，但可能激活多个能力相近的角色，同时遗漏关键能力。本文将选择目标改为：在满足硬约束和并发预算的条件下，覆盖当前任务所需能力，减少重复劳动与跨角色交接开销，并控制频繁切换。

成员画像包括能力、工具、Skill、知识权限、模型配置、历史结果、负载和成本。先用已登记资源和权限构造可行分配关系；某项工具只需由承担对应子任务的成员具备，不能要求团队中每个成员都挂载任务所需的全部工具。未授权、不可用或缺乏必需资源的分配在语义评分前排除。

设待覆盖能力集合为 $\mathcal{C}_\tau$，成员 i 对能力 c 的适配度为 $p_{ic} \in [0,1]$，待选团队为 A。能力覆盖定义为

$$\mathrm{Cov}(A,\tau) = \sum_{c\in\mathcal{C}_\tau} w_c \max_{i\in A} p_{ic}, \qquad \sum_c w_c = 1, \tag{4.8}$$

空集覆盖为零。该定义使已经被充分覆盖的能力具有较小新增收益；必须满足的能力另

外设置阈值，不以平均覆盖替代硬性要求。团队目标为

$$F(A, E \mid \tau, S_t) = \alpha \operatorname{Cov}(A, \tau) + \beta \widehat{U}(A, \tau) - \lambda \sum_{e \in E} \widehat{V}_e - \mu \operatorname{Red}(A) - \nu \operatorname{Switch}(A, A_{t-1}), \tag{4.9}$$

其中 $\widehat{U}$ 为预估任务效用，$\widehat{V}_e$ 为边的预估通信开销，Red 为角色职责重复度，Switch 为成员切换代价。各项在固定统计口径下归一化，权重由验证集选择。相似角色并不总是冗余：独立复核时，应保留相应角色并检查证据与工具独立性。

可实施的选择算法　控制器采用有界的边际增益选择：从空团队或可复用成员开始，每次评估加入候选后对覆盖、效用、通信和切换的影响，选择可行且增益最大的成员；硬能力已满足且继续加入无足够收益时停止。随后将成员分配到子任务，依据输入输出契约编译边，再检查依赖、预算、独立复核及并发限制。不满足条件时进行有限修补；仍不可行则返回缺少的能力和资源，不编造成员或强行执行。

G-Designer 展示了任务条件化生成协作拓扑的研究方向 [4]，AgentPrune 将通信图中的冗余作为优化对象 [5]。本文借鉴其优化对象，将显式任务契约保留为执行边界，不直接复用其图生成或剪枝算法。式 (4.9)含有非单调惩罚与依赖约束，有界贪心仅是可解释近似，不声称全局最优或通用近似比。

任务和成员的语义表示可由编码模型产生，候选适配度及预估效用可由配对评分模型学习。启动阶段采用规则、能力目录和带先验的历史统计；积累轨迹后，使用同类任务的验收结果、成本和对照执行训练评分器。仅观察被选成员会形成选择偏差，因此训练集需要受控对照或离线候选评估，不能把未选成员直接标为低质量。图神经网络可作为后续评分器选项，不作为本文架构成立的前提。

事件驱动更新与有界转交　控制器响应新子任务、能力缺口、成员失败、证据冲突和质量修订，仅更新受影响的剩余子图。能力缺口触发受监督 Handoff；运行异常触发 Fallback；证据冲突增加核验分支；验收通过后撤销已结束的活跃边。已经验收且输入版本未失效的产物继续复用。

同一尝试中限制连续转交次数和重复参与。若需要让原成员返工，则创建新的修订尝试，携带失败原因与新增证据；不能以禁止任何成员再次参与来阻断合理返工，也不能无限允许同一组合循环。阶段和全局重规划均设置上限。

质量门采用可编码检查与可选模型评审：

$$Q = w_{\text{rule}} Q_{\text{deterministic}} + (1 - w_{\text{rule}}) Q_{\text{model}}. \tag{4.10}$$

无模型评审时使用独立校准的规则配置，不能把缺失分数当作已获得的模型判断。显式问题未解决时，即使总分较高也不得直接通过。质量门返回通过、修订或重规划，控制器据此更新下一轮任务及活跃关系。

4.2.4　低熵通信架构：协作边上的语义传输

架构定位与跨领域启发　TIDE-Mesh 把通信实现为**部署在 Agent 与 Agent、Agent 与团队、团队与团队逻辑边上的模块链**。对一条 $u \to v$，发送端产物不直接拼入接收者全部历史，而是通过需求编码、胶囊构造、语义选择、保真压缩、接收校验五个模块，再形成 v 的当前输入。模块可以由规则、编码器或模型实现，端口与协议保持一致。

协同感知中的 When2com 研究通信分组与通信时机 [6]；Where2comm 以空间置信信息选择需要共享的关键区域 [7]。本文将这一思路转化为文本协作中的接收者需求和语义单元选择，不使用空间置信图，也不假定视觉网络可直接处理文本。其对应关系是：任务缺口决定何时需要消息，活跃边决定发给谁，验收所需的主张与证据决定发送什么。

“低熵”在本文中表示**在保持任务必要信息的条件下抑制重复、无关与无依据的信息传播**。它是通信架构的设计名称，不代表最小化字符熵；词项相似度也不等于互信息。系统以通信载荷、字段保真、接收后任务质量和错误采纳率评价效果。

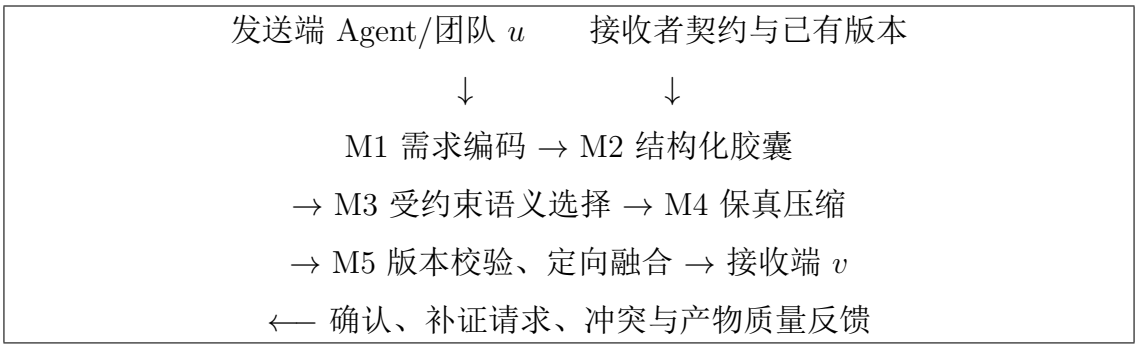


图 4.4: 部署在逻辑协作边上的五模块语义通信架构

M1：接收者需求编码　需求编码器读取边绑定的子任务、接收角色、必需输入、输出契约、可见权限和已确认消息版本，形成接收者需求表示 $d_{v,t}$。需求以契约字段为基础，编码模型可补充语义匹配。编码器不访问接收者隐藏推理过程；未知的接收状态按未知处理，不能假定对方已经知道某个前提。

当边上的阶段完成、依赖产物变化、出现约束撤销或接收者请求补证时，通信被触发。普通说明可以合并或等待下一次交接，但执行依赖、取消和冲突通知属于控制信息，不能因预估贡献偏低而静默消失。

M2：结构化胶囊与证据绑定　胶囊构造器将发送端结果表示为 MessageCapsule，将可筛选信息拆为语义单元，并区分主张、证据、约束变化、产物引用及下一动作。每个单元带来源和验证状态，避免压缩后把“待验证假设”改写为“已确认结论”。完整产物留在归档，胶囊可以只传递接收者当前需要的片段和可访问引用。

表 4.5: MessageCapsule 的通信契约与状态字段

字段类别	内容及用途
寻址与契约	发送端、接收端、边标识、任务及产物版本、消息类型
主张与证据	结论、验证状态、证据摘要、来源及原文引用
保护语义	约束变化、撤销、失败、冲突和执行阻断条件
增量与基准	接收者已确认基础版本、变更字段、删除标记和当前版本
可靠性信息	消息标识、校验摘要、有效期、确认状态和压缩血缘
预算与反馈	载荷预算、实际用量、补证请求及消费结果

M3：受约束的语义单元选择　选择器根据接收者当前任务判断信息是否值得传输。对语义单元 z_j，以任务相关性、证据支持、接收者尚缺信息、时效和重复度估计价值 $\widehat{u}_j$。已有词项或向量相似度可用于发现重复，但不能仅据“文本相似”删除数值变更、相反结论或撤销通知。

设 $x_j \in \{0,1\}$ 表示保留单元，ℓ_j 为长度，$\mathcal{P}$ 为保护单元集合，$\mathcal{K}_v$ 为接收端必需输入集合，选择问题为

$$\begin{aligned} \max_{\mathbf{x}} \quad & \sum_j x_j \widehat{u}_j - \rho \sum_{j<k} x_j x_k \widehat{r}_{jk} \\ \text{s.t.} \quad & \sum_j x_j \ell_j \le b_e, \qquad x_j = 1 \quad (j \in \mathcal{P}), \\ & \mathrm{Cover}(\mathbf{x}, \mathcal{K}_v) = 1. \end{aligned} \tag{4.11}$$

$\widehat{r}_{jk}$ 度量内容重复，覆盖约束要求必要主张连同证据或明确的缺失说明一起交付。若保护语义及必需输入本身超出预算，则该约束集不可行：转为保留完整性的小消息分包、可访问原文引用或显式补证，不悄悄丢弃保护字段。无法满足接收前提时，接收者等待或暂停。

工程路径可采用保护单元优先、按边际价值和长度选择其余单元的近似算法。预估价值可以由轻量模型学习，但必须使用固定任务质量口径校准。新颖性只描述接收者是否见过相关信息，不能证明其正确。

M4：字段感知压缩与保真检查　压缩器对被选中的说明文字去重、摘要和缩短，优先保留事实值、单位、时间、约束变更、验证状态及来源关系。模型可以压缩描述字段，符号检查器核对受保护字段及必需引用；压缩失败时回退到未改写的结构化字段或缩小普通说明范围。

字段一致不保证所有自然语言含义均一致。因此对高风险结论还需执行来源核验或独立复核，并记录压缩前后内容及父消息标识。完整原文在归档中保留，使接收者能按需展开证据，而不是依赖不可追溯的短摘要。

M5：版本校验、定向融合与反馈　接收模块检查权限、任务版本、有效期、消息标识和校验摘要；重复消息不重复提交业务状态。增量必须相对**该接收者已确认的基准**计算。若发送端上一次消息被筛除或未确认，不能继续以其发送版本作为接收端基础版本。

设发送者 u 对接收者 v 的已确认视图为 $\widehat{S}^{b}_{u,v}$，待发送视图为 S^{k}_{u}，则

$$\Delta^{b\to k}_{u,v} = \mathrm{Diff}(\widehat{S}^{b}_{u,v}, S^{k}_{u}). \tag{4.12}$$

接收者校验基础版本后合并增量并返回确认；缺少基础版本时请求完整快照或缺失增量，不能直接套用。删除标记只作用于该业务视图；全局共享状态仍按第4.1节的版本提交规则更新。

确认投递和确认已消费分别记录。运行恢复时同时核对信箱、确认进度和基础视图；在可能重复投递的条件下，通过消息去重和提交幂等性避免重复生效，不宣称普遍的端到端恰好一次执行。控制通知过期后应重新获取当前有效状态，不能因 TTL 到期就继续使用旧约束。

校验后的语义内容进入接收者当前输入，协议元数据保留在运行记录中。接收者可以返回已足够、缺少证据、版本冲突或任务不再需要等反馈；来源正确性仍需独立检查，接收确认不等于事实已验证。

4.2.5　时态证据记忆与跨模块一致性

时态证据层记录“哪个来源在什么时间、基于什么证据提出了哪个版本的事实”。通信胶囊先进入消息记录或候选事实区，只有完成相应验收后，才按第4.1节的准入规则形成 TASK、PROJECT 或 GLOBAL 记忆。这样，消息到达、共享状态提交和经验入库成为三个有区别的事件。

事实记录包含实体、属性、取值、有效时间、记录时间、证据和验证状态。同一槽位在不兼容的时间范围内取不同值时，保留冲突供核验；明确的新版本可以替代旧版本，但历史不得被无痕覆盖。自然语言矛盾不能只靠字符串比较判定。

当事实或记忆被撤销时，系统沿已记录的消费依赖向受影响接收者发送保护通知，使其停止使用相关版本，并由共享状态管理器决定补证或局部重执行。通信模块负责可靠传递撤销信息，TIDE-Anchor 负责管理事实和产物依赖，拓扑控制器负责为修复任务重新组织成员，避免多个模块分别决定同一产物的权威状态。

4.2.6　核心算法、实现边界与评价方法

设候选数为 N，最大激活成员数为 K，能力维度为 d，一次候选效用及局部边估计成本为 T_s。直接实现的边际选择成本约为 $O(KN(d+K+T_s))$；任务图编译后的依赖检查为 $O(|V|+|E|)$，有限图修补需单独计入。该方法比单独排序开销更高，其价值应由任务质量和总执行成本检验。

Algorithm 3: 任务契约驱动的活跃协作图编译

Input: 共享状态 S_t，事件 e_t，候选成员池 $\mathcal{A}$，预算 b

Output: 可执行协作图或不可行原因

更新受影响的剩余任务，确定能力、输入输出和验收条件;
按资源、权限和可用性过滤不合法的任务 –成员分配;
以边际贡献选择互补成员，检查必需能力与并发限制;
根据产物依赖生成交接、汇聚和复核边，绑定通信契约;
检查版本内无环、资源预算与复核独立性，执行有限修补;
if 仍不可行 **then**
    return 缺少的能力、资源或契约条件;
执行就绪子任务，接收质量、失败和能力缺口事件;
关闭结束或失效边；在尝试预算内更新剩余子图;
return 图版本及运行记录;

Algorithm 4: 协作边上的接收者条件化语义通信

Input: 发送端结果 y_u，有效边 $e = (u, v)$，已确认视图 $B_{u,v}$

Output: 接收结果、补证请求或阻断原因

M1：读取接收者契约、缺失输入和已确认版本;
M2：构造带来源、验证状态和保护语义的胶囊;
M3：在预算内选择语义单元，保留必需信息与依赖;
if 必要字段无法装入预算 **then**
    采用分包或引用；无法完整交付则阻断;
M4：压缩普通描述，检查保护字段和引用，失败则回退;
M5：检查权限、有效版本及消息去重；核对增量基础版本;
if 基础版本不一致 **then**
    请求快照或缺失增量;
else
    定向写入信箱，按契约融合，记录确认及消费状态;
反馈信息缺口、冲突和验收结果，更新接收者已确认视图;

设单条消息具有 J 个语义单元、预算离散值为 B。不含成对重复项的背包动态规划成本为 $O(JB)$；完整式 (4.11)含成对项和覆盖约束，不能直接套用这一复杂度。工程近似若缓存成对相似度，其预处理为 $O(J^2T_s')$，其中 T_s' 为单次相似度成本；接收者数和核验模型调用另计。总载荷按实际投递累计：

$$V_{\text{comm}} = \sum_{m \in \text{Delivered}} \text{Size}(m). \tag{4.13}$$

分包、确认、补证、重传和快照都应计入相应通信统计，不以潜在边数直接替代实际 Token 开销。

现有工程提供资源硬过滤、多指标 Top-K、受监督转交与替补，以及图节点边界的胶囊门控、压缩和状态增量。能力互补的图编译、统一成员端口、语义单元选择、接收者确认基准和撤销传播构成本节扩展设计，须在后端接入后验证。原有规则路径可以作为对照和冷启动回退，不据其测试结果宣称扩展设计已经通过实验。

拓扑实验应对比固定流水线、固定团队、原 Top-K 与能力互补图编译，分别观察任务完成、能力缺口、切换与编排耗时。通信实验在相同任务和拓扑下，对比原文传递、结构化胶囊、语义选择和完整五模块链，报告含模块调用开销的 Token、时延、证据保留、压缩失真及错误采纳率。丢包、版本错位、重复投递和撤销通知用于验证协议行为，不能以消息变短直接推断任务质量提高。

4.3　TIDE-Placement：端边云自适应推理调度

4.3.1　设计目标与约束决策架构

端边云放置需要处理长程任务中的跨步骤资源取舍。将某个复杂节点放到高能力资源上，可能提高当前结果质量，却消耗后续节点所需的时间与费用预算；在本地快速响应，也可能因结果不足而触发更多修订。因此，资源选择不仅依赖当前任务复杂度，还应考虑剩余任务、可用预算、服务负载以及既往尝试的执行反馈。

本文将 TIDE-Placement 组织为**硬可行域约束下的学习型资源调度架构**。可信域、权限、模型能力和当前可用性先确定允许动作，策略再在允许范围内优化任务质量与累计资源消耗。其建模借鉴约束强化学习将收益与约束预算分开定义的思想 [8]；具体采用有限时域约束决策过程与拉格朗日 Actor–Critic 设计，不将其等同于 CPO 算法，也不继承后者的理论保证。

架构包括状态编码器、可行域过滤器、策略与价值评估器、约束预算控制器、执行反馈器五个部分。BGE 类编码器与 MLP 可分别承载任务语义表示和策略网络；规则调度器作为冷启动及学习策略不可用时的回退。只有依据奖励和约束轨迹更新策略，才能称为强化学习，已有监督分类路由不能仅通过改写公式获得这一性质。

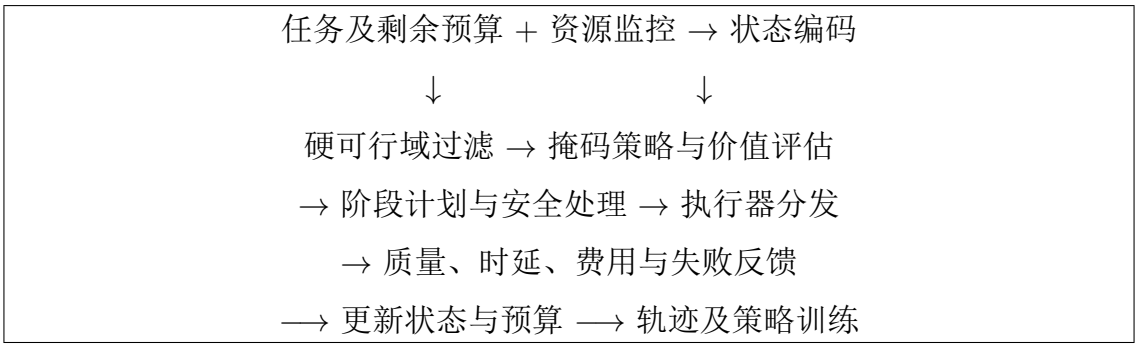


图 4.5: 硬约束与累计预算共同作用的学习型调度架构

4.3.2　任务、资源与调度动作的统一表示

状态构造　以一次可调度的推理请求或失败后的重新申请为决策步 t，任务结束、不可恢复失败或预算终止为回合结束。有限时域上限为 H，包含重试次数，避免通过无限等待规避任务失败。状态表示为

$$s_t = \langle z_t, p_t, \omega_t, b_t, h_t, v_t \rangle, \tag{4.14}$$

其中，z_t 为任务语义及当前输入的表示，p_t 为任务画像，ω_t 为资源状态，b_t 为费用、时间和尝试等剩余预算，h_t 为必要历史统计，v_t 为计划及约束版本。使用共享状态的最小必要投影，不将整个长期对话输入调度器。

任务画像包含实时性、敏感度、复杂度、任务类型、预期产物质量和输入长度；显式元数据优先于模型推断，缺失字段由门控器补充。资源状态记录端点健康度、可用模型、上下文容量、负载、队列、近期时延、限流和错误统计。可信属性来自部署和授权配置，不能把“云”天然视为不可信，也不能把“边缘”天然视为可信。

资源画像应针对具体端点和模型配置。端、边、云只是位置标签：设备可能更快但能力有限，边缘可能适合内部数据处理，云侧可能提供更高能力模型，实际比较以部署参数和探测结果为准。参考时延与成本权重是配置值，不能写成实测硬件收益。

状态未必完整描述环境。网络和服务负载可能不可观测，式 (4.14)是用于决策的近似充分统计；必要时加入滑动窗口或历史编码。状态表示是否足以支持稳定决策，需要通过分布变化和负载扰动实验验证。

动作与阶段计划　资源动作定义为

$$a_t = \langle r_t, m_t, \xi_t \rangle \in \mathcal{A}(s_t), \tag{4.15}$$

其中 r_t 是具体资源端点，m_t 为可用模型配置，ξ_t 为经批准的推理参数。动作空间规模需受到限制，以便估计质量、时延与费用。无可行推理资源时，控制器返回等待、拆分请求或需要复核的明确状态；这些处理受时限和次数约束，不作为任意长的无成本动作。

调度动作进一步生成任务级阶段计划 Π_t：任务画像构建、必要的可信预处理、一次主推理及可选产物检查。典型路径为

$$\text{本地画像} \rightarrow \text{可信预处理（按需）} \rightarrow \text{选定端点主推理} \rightarrow \text{质量检查}. \tag{4.16}$$

本节讨论的是任务阶段放置，不等同于将单个神经网络的层或张量切分到多台设备。阶段名称和资源位置显式记录，使计划可以被执行器和审计模块共同读取。

4.3.3　硬可行域与有限时域约束建模

不能被收益补偿的硬约束　可行域过滤器先检查动作是否合法：

$$\mathcal{A}_{\text{safe}}(s_t) = \left\{ a \mid \begin{array}{l} \text{Available}(a, s_t) = 1, \\ \text{Authorized}(a, s_t) = 1, \\ \text{TrustOK}(a, s_t) = 1, \\ \text{CapabilityOK}(a, s_t) = 1 \end{array} \right\}. \tag{4.17}$$

能力条件包括上下文容量、必要模型接口和阶段输出要求。对必须达到的能力门槛严格过滤；可选的质量偏好留给策略优化。将高复杂任务送到能力不足的本地模型必须经过显式任务降级或拆分，不能作为满足原始要求的成功回退。

敏感载荷仅能流向被当前策略授权的处理域。若采用脱敏以允许外部处理，必须验证输入、消息、附件及请求元数据的实际发送载荷均符合相应策略，不能仅因存在脱敏阶段就自动允许外传。安全预处理后和请求发送前再次核验，以处理策略、资源和载荷在调度后发生变化的情况。

敏感性与授权策略 + 具体端点可信属性
→ 生成允许动作 → 策略选取候选
→ 可信预处理与实际载荷核验
→ 通过后执行；不满足则阻断、改选或复核

图 4.6: 独立于学习收益的可信域与发送前校验

收益与约束成本　有限时域约束马尔可夫决策过程用状态、动作、未知转移、收益和约束成本描述调度。收益 r_t 由可核验任务进展及阶段质量构成，终止时加入最终交付评价。重复完成同一产物不能重复获得进展奖励；提前放弃按未完成任务评价，不能仅因省费用而被评为优质策略。模型评审可补充语义质量，但需结合规则和真实证据校准。

定义费用成本 $c_{\text{fee},t}$、经过时间 $c_{\text{time},t}$ 及失败尝试 $c_{\text{fail},t}$。费用包含实际调用及安全处理、重试等可计费开销；时间按墙钟区间的增量累计，并行运行不能简单累加各节点耗时作为端到端时延。能耗仅在有测量或经过校准的估计器时加入，不能以费用权重冒充功耗。

策略目标为

$$\begin{aligned} \max_{\pi} \quad & J_R(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{H-1} r_t\right] \\ \text{s.t.} \quad & J_i(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{H-1} c_{i,t}\right] \le d_i, \quad i \in \{\text{fee}, \text{time}, \text{fail}\}, \\ & \pi(a \mid s) = 0, \quad a \notin \mathcal{A}_{\text{safe}}(s). \end{aligned} \tag{4.18}$$

d_i 为对应任务类别的预算。若不同任务预算不同，应将预算纳入状态并分别报告可行性，不用混合平均值掩盖困难任务超限。训练可按各任务预算归一化成本，最终报告仍使用实际费用、时间和次数。

这里的累计约束是**期望约束**，不能保证每条任务轨迹都不超限。运行时另设剩余预算检查：若可预估开销将超过剩余额度则阻断或拆分，并对单次调用设置费用或超时上限。网络时延和失败具有随机性，预估不能提供确定性截止时间保证；若业务要求严格实时约束，需要额外的资源预留和可证明的最坏耗时控制。

不可行任务的处理　当可信资源不可用、必需能力缺失或预算不足时，优先生成不可行原因，而不是在多个禁止动作中选取“分数最高者”。允许等待的任务设置最大等待窗口，等待时间计入约束成本；可以拆分的任务交还规划器重构；需授权或需求降级的任务暂停。任何新授权或降级都更新状态与约束版本，再重新求取可行域。

4.3.4　策略网络、价值评估与约束学习

状态编码与掩码策略　状态编码器将任务语义向量与画像、资源状态、剩余预算和历史统计拼接。BGE 类模型可提供语义向量，MLP 输出候选动作分数 $f_\theta(s,a)$。对安全可行域内的动作进行归一化：

$$\pi_\theta^{\text{safe}}(a \mid s) = \begin{cases} \dfrac{\exp f_\theta(s,a)}{\sum_{a' \in \mathcal{A}_{\text{safe}}(s)} \exp f_\theta(s,a')}, & a \in \mathcal{A}_{\text{safe}}(s), \\ 0, & \text{其他动作}. \end{cases} \tag{4.19}$$

可行域为空时不计算该分布，而进入不可行任务处理。掩码仅对已编码且使用最新数据检查的条件有效，不替代策略覆盖检查或发送前核验。

价值评估器分别估计收益价值 $V_R(s)$ 与各成本价值 $V_i(s)$。独立输出使费用、时间和失败预算具有可解释的预测，不需要将所有指标固定揉成单个静态权重分数。预测用于学习和决策，其准确性需要在独立轨迹上校准。

拉格朗日 Actor–Critic 更新　对式 (4.18)构造

$$\mathcal{L}(\theta, \boldsymbol{\lambda}) = J_R(\pi_\theta) - \sum_i \lambda_i \left(J_i(\pi_\theta) - d_i\right), \qquad \lambda_i \ge 0. \tag{4.20}$$

策略参数沿收益增加方向更新，约束乘子在观测成本超预算时上调。用收益与成本优势估计构造策略优势：

$$\widehat{A}_t^{\lambda} = \widehat{A}_{R,t} - \sum_i \lambda_i \widehat{A}_{i,t}. \tag{4.21}$$

更新形式为

$$\begin{aligned} \theta &\leftarrow \theta + eta_\theta \widehat{\nabla}_\theta \mathcal{L}, \\ \lambda_i &\leftarrow \left[\lambda_i + \eta_\lambda \left(\widehat{J}_i - d_i\right)\right]_+ . \end{aligned} \tag{4.22}$$

其中 $\widehat{J}_i$ 应由完整回合或一致的截断引导估计得到，不能直接拿一个决策步的成本与整项任务预算比较。奖励与成本的尺度分别固定或归一化，乘子变化用于调整相应预算的约束压力。

训练时按实际掩码后的策略采样并记录动作概率，使用匹配行为策略的轨迹进行 Actor–Critic 更新。若调用前校验拒绝动作，将拒绝原因和时间开销记录为环境反馈；不将规则替换后的动作伪记为原策略采样结果。推理部署可使用确定性选择，但需独立评价该部署策略，因为训练期随机策略的期望约束不自动适用于取最大分数的策略。

拉格朗日方法可以出现振荡、估计误差与约束违反，需要通过独立验证和运行时硬过滤控制风险。这里提出的是可实现的学习与控制架构，不声称有限样本下收敛到全局最优，也不声称训练阶段始终满足全部累计预算。

训练数据与逐步接入 首先由现有规则、分类路由和受控对照执行形成启动轨迹，记录任务条件、候选集、实际动作、质量证据、费用、时间和失败。监督预训练可让策略模仿合理路由或预测各候选收益，但这一阶段仍属于监督学习。

随后在沙箱任务或可控试验环境中，用掩码后的策略收集完整回合，依据式 (4.20)更新策略、价值网络和约束乘子。仅依赖旧策略日志训练时，必须处理动作覆盖和离策略偏差；本节采用先预训练、再受控在线采样的接入路径，不将直接套用在策略梯度上的旧日志当作有效强化学习训练。

训练、验证和测试按任务来源及时间区间划分；资源故障和网络扰动在测试中独立设置。调度器只读取当前可见信息，最终质量仅作为事后奖励，不能泄漏到当次状态。需要保存模型、画像规则、约束预算和策略版本，确保调度轨迹可复核。

4.3.5 执行器分发、失败反馈与运行时接入

选定动作被封装为 SchedulingDecision 和 ResourceAllocation，包含具体端点、模型、策略版本、动作掩码、阶段计划、安全检查及选择依据。ExecutorRegistry 据此调用本地、边缘 HTTP 或云侧兼容执行器，并统一返回成功状态、实际用量、耗时、错误类型和重试属性。实际推理位置以执行记录为准，不能仅用计划中写入的层级当作已发生的部署。

请求发出后，资源监控器更新端点级的健康度、限流与近期时延。限流端点进入受控冷却窗口；下一次调度读取新状态并再次过滤和选取。连接错误不一定意味着同层所

有端点均失效，监控粒度应与动作中的具体端点对应。

公开高复杂请求 → 选择云端点 1 → 收到 429
→ 计入本次费用、时间与失败成本
→ 更新端点 1 状态，重算剩余预算及可行域
→ 选择另一可行端点，或因能力/预算不足退出

图 4.7: 将失败计入约束反馈后的有限重调度示例

可重试错误在次数、剩余预算和截止时间允许时重新调度；不可重试错误直接返回处理原因。失败尝试也消耗预算，不能只统计最终成功调用。敏感任务的云端失败回退和边缘失败回退均需重新检查授权，恢复路径不能绕过原始约束。

运行时由 HookManager 在节点执行前获取共享状态与资源状态，调用调度接口并写入分配结果。节点结束后回传产物验收和实际消耗；任务最终质量可能延迟到质量门或结果汇聚后才获得，应通过任务和尝试标识归入对应轨迹，避免把当前 HTTP 成功直接当作任务成功奖励。

表 4.6: 学习型调度轨迹的主要记录

类别	主要内容
决策输入	任务与资源画像、计划版本、剩余预算和可见历史
可行域	候选动作、排除原因、可信域及能力检查结果
策略输出	策略版本、动作概率或部署分数、价值预测和约束乘子
实际执行	具体端点及模型、安全载荷校验、阶段计划和尝试标识
执行反馈	质量证据、实际费用、墙钟时间、错误与失败成本
预算与终止	剩余额度、超限事件、重试原因、不可行或暂停原因

4.3.6　核心算法、实现边界与验证方案

设候选动作数为 N，任务编码成本为 T_{enc}，策略前向成本为 T_{π}，资源检查及预算过滤为 $O(N)$，单次决策成本为 $O(T_{\text{enc}} + T_{\pi} + N)$。最多 K 次重调度时，结构性决策开销相应乘以 K；安全处理、质量评审、网络与模型推理耗时另计。训练成本由轨迹采集量、回合长度、模型规模和更新次数决定，不能用在线选择复杂度替代。

现有工程包含任务画像、规则与分类路由、可信域策略、执行器分发、资源监控和有限重调度。有限时域 CMDP、掩码策略网络、成本价值网络、约束乘子训练及完整奖励归因为本节扩展设计。BGE+MLP 可复用为网络结构，但需要完成策略训练与部署验证，才可报告约束强化学习的性能收益。

验证应在相同模型、任务与故障轨迹下，对比固定端点、规则调度、原有分类路由、无约束学习及完整约束学习。报告最终任务质量、费用、端到端时延、失败与期望

Algorithm 5: 硬可行域约束下的资源调度与执行

Input: 当前任务与共享状态 S_t，资源监控 Ω_t，策略 π_θ

Output: 执行结果、下一状态及决策轨迹

编码任务、资源、历史与剩余预算，得到 s_t;
按授权、可信域、能力、可用性构造 $\mathcal{A}_{\text{safe}}(s_t)$;
结合剩余预算排除不允许发起的调用;
if 无可行动作 **then**
    返回有界等待、拆分或暂停原因;
在实际可行域内按当前部署规则选择动作 a_t;
生成阶段计划，对实际发送载荷和最新资源状态再次校验;
if 校验失败 **then**
    记录原因与消耗，返回重新决策请求;
执行请求，记录实际用量、耗时和失败;
更新资源监控、预算、任务状态和质量反馈关联;
return 结果及 $(s_t, a_t, r_t, \mathbf{c}_t, s_{t+1})$ 轨迹记录;

Algorithm 6: 累计预算约束下的有限重调度

Input: 请求 q，最大尝试次数 K，剩余预算 b

Output: 结果或明确终止原因

for $k \leftarrow 1$ **to** K **do**
    读取最新状态与约束，检查等待窗口及剩余预算;
    调用算法 5并记录本次实际消耗;
    if 结果通过当前验收 **then**
        return 结果;
    if 错误不可重试或预算不足或无可行资源 **then**
        return 终止原因;
    将失败写回端点状态，更新下一次调度输入;
return 尝试预算耗尽及全部记录;

预算违反，并另外报告单条轨迹超限率和硬约束违规。改变预算阈值形成质量 –成本和质量 –时延曲线，检查策略是否随预算调整，而不是只在一个固定权重下表现较好。

消融包括去掉剩余预算状态、约束乘子、硬动作过滤、失败反馈和质量奖励。硬约束消融仅在合成数据与隔离环境中验证行为。无可信候选、模型容量不足、限流、预算突然缩减和质量反馈延迟用于检验不可行退出及分布变化。所有扩展收益以实际完成的对照实验为依据，既有规则分支测试不作为学习收敛或最优性证据。

4.4 神经符号协同、动态拓扑与自主恢复闭环

TIDE-Anchor、TIDE-Mesh 与 TIDE-Placement 围绕同一任务过程发挥作用：账本与历史证据提供决策上下文，动态团队选择承担当前任务的成员，资源调度为启用自动选择的调用确定推理服务。执行结果再通过质量反馈、状态更新和失败记录影响后续步骤。

三项机制分别管理不同对象：上下文账本维护任务信息，团队控制器维护成员参与关系，调度器维护资源分配。它们通过共享状态、节点配置和执行钩子衔接，不要求所有工作流同时启用全部组件。

4.4.1 计划中间表示与反例驱动校验

神经侧生成候选计划，符号侧检查已编码约束，反例驱动有限修订。系统以 `PlanIR` 承载目标、子任务、硬软约束、资源、验收条件、风险、版本与来源；子任务记录依赖、时长、角色、资源需求及预期条件。`LLMPlanGenerator` 将模型返回的 JSON 解析为这一中间表示，确定性生成器提供离线对照路径。结构化表示使自然语言规划结果能够进入统一的校验和运行观测接口。

`SymbolicPlanValidator` 首先检查目标非空、任务标识唯一、依赖存在性、自依赖、负时长和依赖环。确定性后端进一步依据依赖形成最早调度，检查角色可用性、单任务资源需求以及已编码的先后、互斥、截止时间、硬策略和必需证据条件；可选 Z3 后端对依赖、先后、互斥和截止时间建立时序约束，返回可行调度或不可满足核。校验结果保留后端名称、违反项和反例，使修订具有明确定位依据。

算法 7概括了这一过程。`NeuroSymbolicReasoner` 通过生成器、校验器与修订器接口组织闭环。失败反例写入 Capsule 的 `constraint_delta` 并标记冲突，保留约束标识、原因和见证信息。修订器据此更新计划并递增版本，系统记录每次修订；默认最多修订 2 次。已提供的确定性修订器处理依赖环和缺失依赖，其他修订策略通过 `PlanRepairer` 接入。反例进入结构化反馈后，可沿通信层的约束与冲突保护规则保留修订依据。

启用推理组件时，运行钩子在运行初始化路径写入计划、校验结果和修订记录；校验失败标记 `plan_validation_paused`。节点上下文按计划映射注入子任务、硬约束

与计划版本。这一设计将计划有效性从自由文本说明转化为可读取的运行状态，并为 TIDE-Anchor 提供可追溯的计划依据。

Algorithm 7: 反例驱动的有限计划校验与修订

Input: 目标 g，上下文 c，生成器 F，校验器 V，修订器 R，修订上限 J

Output: 计划 p，校验结果 v 与修订轨迹

$p \leftarrow F(g, c)$;
for $j \leftarrow 0$ **to** J **do**
  $v \leftarrow V(p)$;
  if v 通过 **then**
    return p, v 与修订轨迹;
  将 v 的反例封装为约束变化与冲突 Capsule;
  if $j = J$ **then**
    break;
  $p \leftarrow R(p, v.\text{counterexamples})$，更新版本与修订记录;
return p, v 与修订轨迹，由运行钩子记录校验暂停状态;

设计划包含 n 个子任务、e 条依赖和 c 条约束，显式结构检查与依赖遍历成本为 $O(n+e)$，确定性约束检查另计资源需求条目与 c 的扫描成本。若单次生成、校验和修订成本分别为 T_F、T_V 和 T_R，则闭环成本为 $O(T_F + (J+1)T_V + JT_R)$；Z3 路径的 T_V 由约束规模与求解过程决定，模型生成与修订调用单独计费和计时。

4.4.2 质量门控与任务驱动的拓扑协同

计划校验面向执行前的结构与约束，质量门面向执行后的产物与证据，两者共同连接规划和交付。质量门读取当前 TODO、团队结果、真实运行来源与工具记录，并按预设检查非空输出、成员执行状态、来源数量和代码测试证据。配置评审模型时，再结合原始目标、验收条件和运行来源判断完成度、自洽性与证据支持度。规则分与模型分按第4.2节的 Q 组合，默认规则权重为 0.45、通过阈值为 0.75；通过条件同时要求问题列表为空，使高分与显式问题检查共同约束阶段验收。

质量反馈通过规划器更新任务账本：`pass` 保存阶段产物和证据，`revise` 携带返工指令再次执行当前任务，`replan` 在全局重规划预算内保留已完成结果并重建剩余 TODO。TIDE-Anchor 稳定供给目标与历史依据，TIDE-Mesh 据更新后的任务描述、资源需求和反馈重新过滤、评分与选择成员，TIDE-Placement 据当前请求及资源状态安排推理阶段。因而，同一返工流程能够同时调整证据输入、参与角色和执行资源。

动态拓扑以任务状态和运行事件为驱动，以控制骨架与资源约束为边界。新 TODO 触发初始稀疏委派，新增能力需求触发受监督 Handoff，成员异常触发 Fallback，质

量修订与重规划触发下一轮重新选人。每轮关系随任务生命周期生成并记录，候选角色定义保持可复用。需求变更经任务状态或修订指令显式进入该闭环，约束变化通过 Capsule 保护字段和账本持续传递，从而把语义变化落实为可解释的协作调整。

4.4.3 异常分级恢复与闭环终止

恢复动作按失败对象分层处理：成员执行异常由 TIDE-Mesh 选择可行备用成员；可重试的推理服务异常由 TIDE-Placement 写回资源状态并重新申请资源；阶段质量问题通过修订和剩余任务重规划处理；执行中断由 TIDE-Anchor 的上下文和图检查点提供恢复依据。运行钩子还依据配置记录重试、模型或工具回退、备用节点、补偿、检查点恢复请求与资源迁移等动作，交由对应执行路径处理。各层共享失败原因和尝试记录，使局部异常能够为后续决策提供信息。

自主执行以明确预算和可行条件约束：团队限制并发、转交及重复参与，质量流程限制阶段尝试和全局重规划，推理 Runner 限制调用尝试次数，计划推理器限制修订次数。全部必要 TODO 通过后进入结果汇聚；预算超限、计划校验失败、重规划预算耗尽或需要人工复核时，记录暂停或失败及其原因。无人工干预的演示以自动恢复策略已配置且恢复条件满足的任务为范围，同时保留受控退出路径与完整轨迹。

4.4.4 运行观测与赛题评价对应

运行观测按任务、节点和尝试记录关联事件，重点保留决策输入、处理结果与失败原因。各类记录与评价指标的对应关系见表 4.7。

表 4.7: 核心机制的运行记录与评价指标

机制	主要记录	评价内容
TIDE-Anchor	账本变化、检索证据、预算判定、漂移与恢复记录	目标与约束保持、证据使用、上下文用量、恢复成功率
TIDE-Mesh	候选评分、委派与转交、替补、消息门控及压缩记录	任务完成率、活跃关系、通信量、字段保真与错误传播
TIDE-Placement	任务画像、资源状态、分配结果与执行尝试	实际层级、执行时间、调用成本与回退结果
综合闭环	任务项状态、质量报告、最终产物与人工介入	最终交付质量、任务完成率与自主执行程度

运行证据按照“机制触发、阶段验收、最终交付”三级关联：事件确认处理分支，质量报告确认当前阶段的验收结果，最终产物与任务标准确认整体交付；资源分配与实际执行位置逐次核对。

实验评估应进一步关联任务配置、模型版本、输入数据、原始轨迹与最终产物，并区分上下文估算、通信协议载荷和模型实际 Token 用量。具体实验设置与结果统一在第六章报告。

适用范围　神经符号结论限定于 PlanIR 中已编码且被所选后端实际检查的约束。确定性后端按最早调度检查，Z3 后端搜索所编码的时序可行性，二者检查集合与调度语义分别记录；角色、资源和证据检查按实际后端及运行门控核对。PlanIR 中的前置条件、效果和软约束等描述字段用于表达与扩展，自动证明范围以校验实现为准。计划校验通过、来源条目有效与业务事实正确分层评价。

推理组件按配置启用，现有 Hook 在运行初始化时校验一次，已有 PlanIR 时复用相应状态；质量门的 TODO 重规划属于运行时反馈路径。变更后计划重新送入符号校验需由相应调用入口显式触发，验证轨迹据实记录。恢复请求事件与实际恢复完成分别核验，检查点版本、消息连续性和外部副作用按各机制的适用范围处理。实验报告关联任务配置、模型版本、输入数据、原始轨迹和最终产物；数百或数千步、跨领域泛化以及硬件性能收益以实际完成的实验覆盖为依据。

4.5　本章小结

本章介绍了 TIDE-Swarm 的三项核心机制。TIDE-Anchor 通过账本、分层记忆与原文归档维护任务信息，结合预算检查、漂移检测和检查点支持持续执行；TIDE-Mesh 根据任务需求选择异构成员，通过转交、替补与结构化通信组织协作；TIDE-Placement 结合任务画像、可信域策略和资源状态选择推理服务，并利用执行反馈进行有限重调度。

神经符号协同通过计划中间表示、显式校验、反例修订和质量反馈贯穿上述机制，将任务语义、动态拓扑与资源决策接入同一可观测闭环。其设计优势在于稳定保存目标与证据、按需组织异构能力、保护约束变化并利用失败反馈调整后续执行；核心算法、适用范围及计算开销共同构成平台实现和实验评估的依据。下一章介绍相关机制在平台中的集成及典型任务应用。

参考文献

[1] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[2] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. Magentic-One: A generalist multi-agent system for solving complex tasks. Microsoft Research technical report, 2024.

[3] Microsoft. AI agent orchestration patterns. Azure Architecture Center. Accessed 2026-11-03.

[4] Guibin Zhang, Yanwei Yue, Xiangguo Sun, Guancheng Wan, Miao Yu, Junfeng Fang, Kun Wang, Tianlong Chen, and Dawei Cheng. G-Designer: Architecting multi-agent communication topologies via graph neural networks. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 76678–76692, 2025.

[5] Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. Cut the crap: An economical communication pipeline for LLM-based multi-agent systems. *arXiv preprint arXiv:2410.02506*, 2024.

[6] Yen-Cheng Liu, Junjiao Tian, Nathaniel Glaser, and Zsolt Kira. When2com: Multi-agent perception via communication graph grouping. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4106–4115, 2020.

[7] Yue Hu, Shaoheng Fang, Zixing Lei, Yiqi Zhong, and Siheng Chen. Where2comm: Communication-efficient collaborative perception via spatial confidence maps. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[8] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 22–31, 2017.